Bài 2.11 – Gradient Boosting: XGBoost, LightGBM

> **Giai đoạn 2 – Machine Learning cổ điển** · Thời lượng: 2 buổi · Hình thức mở đầu: *So sánh ví von*

🎯 Sau bài này bạn sẽ

- Hiểu Boosting: các cây **nối tiếp nhau sửa lỗi**, khác với Random Forest (các cây độc lập).
- Tự tính tay 3 bước Gradient Boosting.
- Điều khiển được `learning_rate`, `n_estimators`, và dùng **early stopping**.
- Dùng thành thạo **XGBoost** và **LightGBM**, hai thư viện "vô địch Kaggle" cho dữ liệu dạng bảng.

Chia buổi:

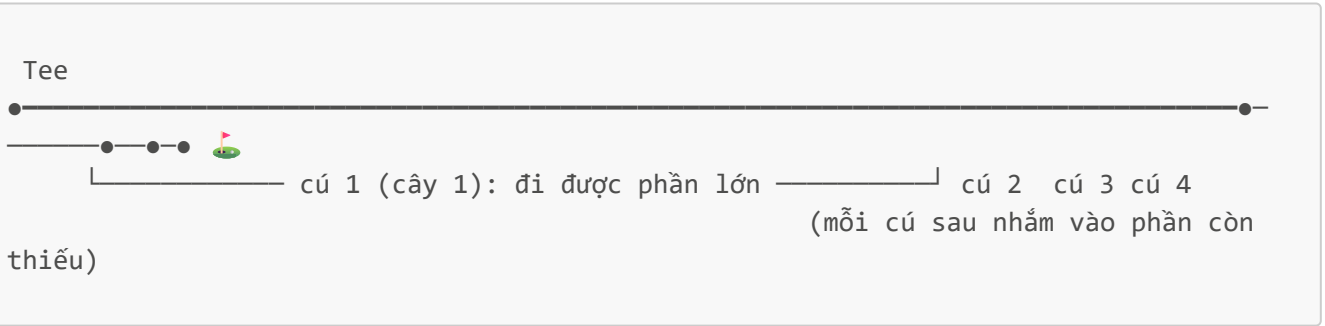
- **Buổi 1:** Mục 1 → 4 (ý tưởng, tính tay, learning rate, số cây).
- **Buổi 2:** Mục 5 → 9 (XGBoost, LightGBM, early stopping, cột chữ, bài tập).

---

1. So sánh ví von: Đánh golf

Bạn đang chơi golf. Lỗ golf cách 300 mét.

- **Cú 1 (cú phát bóng):** đánh thật mạnh về phía lỗ. Bóng đi được 250 m, còn cách lỗ **50 m**.
- **Cú 2:** không đánh lại từ đầu. Bạn đánh **từ chỗ bóng đang nằm**, nhắm vào **khoảng cách còn thiếu** (50 m). Bóng còn cách lỗ **8 m**.
- **Cú 3:** gạt nhẹ để sửa nốt 8 m. Còn **1 m**.
- **Cú 4:** gạt bóng vào lỗ.



Gradient Boosting làm đúng như vậy:

- Cây 1 đưa ra một dự đoán thô.
- Cây 2 học **phần sai số** mà cây 1 để lại.
- Cây 3 học phần sai số **còn lại** sau cây 1 + cây 2.
- ...cứ thế hàng trăm cây, mỗi cây sửa một chút.

	Random Forest (bài 2.10)	**Gradient Boosting**
Các cây	**Độc lập**, song song	**Nối tiếp**, cây sau sửa cây trước
Mỗi cây	Sâu, mạnh (dễ overfit riêng lẻ)	**Nông**, yếu (thường độ sâu 3–8)
Gộp	Trung bình	**Cộng dồn**

	Random Forest (bài 2.10)	Gradient Boosting
Chủ yếu giảm	Variance	**Bias**
Ví von	Hỏi ý kiến 500 người rồi lấy trung bình	Một người đánh golf, mỗi cú sửa cú trước

---

2. Tính tay Gradient Boosting

2.1. Dữ liệu đồ chơi

6 căn hộ, x là diện tích (chục m^2), y là giá (trăm triệu đồng):

x	1	2	3	4	5	6
y	10	12	20	22	40	44

Mỗi "cây" là một **gốc cây (stump)**: cây chỉ có 1 câu hỏi (độ sâu 1). **Learning rate (η) = 0,5**: mỗi cây chỉ được sửa **một nửa** sai số nó tìm thấy (giải thích ở mục 3).

2.2. Bước 0: dự đoán khởi đầu

Dự đoán mọi căn bằng **trung bình**: (10 + 12 + 20 + 22 + 40 + 44) / 6 ≈ **24,67**.

MSE = trung bình $(y - 24{,}67)^2$ ≈ **168,89**.

2.3. Bước 1

Phần dư (y − dự đoán): −14,67; −12,67; −4,67; −2,67; 15,33; 19,33.

Cây 1 học **phần dư** (không phải học y!). Câu hỏi tốt nhất: "x ≤ 4,5?"

- Nhóm trái (x = 1..4): trung bình phần dư = (−14,67 − 12,67 − 4,67 − 2,67) / 4 ≈ **−8,67**
- Nhóm phải (x = 5, 6): trung bình = (15,33 + 19,33) / 2 ≈ **17,33**

Cập nhật: **dự đoán mới = dự đoán cũ + η × cây 1**

- x = 1..4: 24,67 + 0,5 × (−8,67) ≈ **20,33**
- x = 5, 6: 24,67 + 0,5 × 17,33 ≈ **33,33**

MSE giảm xuống **56,22**.

2.4. Bước 2, 3: tiếp tục sửa phần còn thiếu

```
import numpy as np
from sklearn.tree import DecisionTreeRegressor

x = np.array([1, 2, 3, 4, 5, 6], dtype=float).reshape(-1, 1)
y = np.array([10, 12, 20, 22, 40, 44], dtype=float)
lr = 0.5                                          # learning rate

du_doan = np.full(6, y.mean())                    # bước 0: ai cũng bằng trung
bình
```

```
print("Bước 0:", du_doan.round(2), "| MSE =", round(((y - du_doan) ** 2).mean(),
2))
for buoc in range(1, 4):
    phan_du = y - du_doan                              # phần còn thiếu
    cay = DecisionTreeRegressor(max_depth=1).fit(x, phan_du)   # cây học PHẦN DƯ
    du_doan = du_doan + lr * cay.predict(x)            # cộng một phần sửa chữa
    print(f"Bước {buoc}: phần dư cây học = {phan_du.round(2)} | dự đoán mới =
{du_doan.round(2)}"
          f" | MSE = {((y - du_doan) ** 2).mean():.2f}")
```

Kết quả mẫu:

```
Bước 0: [24.67 24.67 24.67 24.67 24.67 24.67] | MSE = 168.89
Bước 1: phần dư cây học = [-14.67 -12.67  -4.67  -2.67  15.33  19.33] | dự đoán
mới = [20.33 20.33 20.33 20.33 33.33 33.33] | MSE = 56.22
Bước 2: phần dư cây học = [-10.33  -8.33  -0.33   1.67   6.67  10.67] | dự đoán
mới = [15.67 15.67 22.67 22.67 35.67 35.67] | MSE = 23.56
Bước 3: phần dư cây học = [-5.67 -3.67 -2.67 -0.67  4.33  8.33] | dự đoán mới =
[14.08 14.08 21.08 21.08 38.83 38.83] | MSE = 8.51
```

Chỉ với 3 "gốc cây" cực kỳ đơn giản, MSE giảm từ 168,89 xuống **8,51**. Để ý: ở bước 2, cây tự tìm ra câu hỏi **khác** ("x ≤ 2,5?") vì phần sai lớn nhất giờ nằm ở chỗ khác.

2.5. Vì sao gọi là "Gradient"?

Với loss MSE, **phần dư (y − ŷ) chính là hướng ngược gradient** của loss theo dự đoán (bài 1.9: đạo hàm của $(y - \hat{y})^2$ theo ŷ là $-2(y - \hat{y})$). Mỗi cây mới học "đi một bước xuống dốc", giống Gradient Descent, nhưng bước đi là **một cái cây** thay vì một con số. Thay loss khác (Log Loss cho phân loại, bài 2.4), công thức "phần dư" đổi theo, nhưng ý tưởng giữ nguyên.

---

3. Learning rate và số cây

3.1. Số cây (n_estimators)

Dùng ô `nap_du_lieu_nha()` ở bài 2.1. `staged_predict` cho phép xem dự đoán **sau từng cây**.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, r2_score

df = nap_du_lieu_nha()
cot = ["thu_nhap", "km_tp_lon", "phong_moi_ho", "ti_le_phong_ngu",
       "nguoi_moi_ho", "tuoi_nha", "kinh_do", "vi_do"]
Xtr, Xte, ytr, yte = train_test_split(df[cot], df["gia_nha"], test_size=0.2,
random_state=42)

gb = GradientBoostingRegressor(n_estimators=1000, learning_rate=0.1, max_depth=3,
```

```
random_state=42).fit(Xtr, ytr)
mae_train = [mean_absolute_error(ytr, p) for p in gb.staged_predict(Xtr)]
mae_test = [mean_absolute_error(yte, p) for p in gb.staged_predict(Xte)]
for i in [1, 10, 50, 100, 300, 1000]:
    print(f"{i:>4} cây: MAE train = {mae_train[i - 1]:5.1f} | MAE test =
{mae_test[i - 1]:5.1f}")
```

Kết quả mẫu (khoảng 30 giây):

```
   1 cây: MAE train =  74.7 | MAE test =  73.5
  10 cây: MAE train =  54.1 | MAE test =  53.7
  50 cây: MAE train =  36.5 | MAE test =  37.1
 100 cây: MAE train =  32.5 | MAE test =  33.6
 300 cây: MAE train =  27.9 | MAE test =  30.4
1000 cây: MAE train =  22.5 | MAE test =  28.3
```

- Khác Random Forest: **mỗi cây thêm vào làm train tốt hơn**, và nếu thêm mãi, Boosting **sẽ overfit** (khoảng cách train–test ngày càng giãn: 0,6 ở 50 cây → 5,8 ở 1.000 cây).
- Ở đây test vẫn đang giảm tới 1.000 cây, nhưng chậm dần. Cần một cách tự động biết "khi nào dừng": **early stopping** (mục 5.3).

3.2. Learning rate

```
for lrate in [1.0, 0.3, 0.1, 0.03]:
    g = GradientBoostingRegressor(n_estimators=300, learning_rate=lrate,
max_depth=3, random_state=42).fit(Xtr, ytr)
    print(f"learning_rate = {lrate:<4}: MAE train = {mean_absolute_error(ytr,
g.predict(Xtr)):5.1f}"
          f" | MAE test = {mean_absolute_error(yte, g.predict(Xte)):5.1f}")
```

Kết quả mẫu:

```
learning_rate = 1.0 : MAE train =  19.3 | MAE test =  34.0
learning_rate = 0.3 : MAE train =  23.1 | MAE test =  28.9
learning_rate = 0.1 : MAE train =  27.9 | MAE test =  30.4
learning_rate = 0.03: MAE train =  33.1 | MAE test =  34.0
```

learning_rate	Hiện tượng (cùng 300 cây)
1,0	Mỗi cây sửa **toàn bộ** sai số → học quá nhanh, bám cả nhiễu → overfit (train 19,3, test 34,0)
0,3	Tốt nhất ở 300 cây
0,03	Học quá chậm, 300 cây chưa đủ → underfit

Quy tắc vàng: learning_rate nhỏ + nhiều cây thường cho kết quả tốt nhất, nhưng chậm hơn. Learning_rate và số cây **phải chỉnh cùng nhau**: giảm learning_rate một nửa thì thường cần gấp đôi số cây.

Giống golf: đánh từng cú nhẹ, chính xác (learning rate nhỏ) cần nhiều cú hơn, nhưng ít khi đánh vọt qua lỗ.

---

4. Các tham số quan trọng

Tham số	Ý nghĩa	Gợi ý khởi đầu
`n_estimators`	Số cây	Đặt lớn (1.000–5.000) + early stopping
`learning_rate`	Mỗi cây sửa bao nhiêu %	0,03–0,1
`max_depth` / `num_leaves`	Độ phức tạp mỗi cây	3–8 / 15–63
`subsample`	Mỗi cây chỉ học trên x% số dòng ngẫu nhiên	0,7–0,9 (thêm đa dạng, chống overfit)
`colsample_bytree`	Mỗi cây chỉ dùng x% số cột	0,7–0,9
`min_child_samples` / `min_child_weight`	Số mẫu tối thiểu mỗi lá	20 (LightGBM)
`reg_alpha`, `reg_lambda`	Phạt L1, L2 lên giá trị các lá (bài 2.6)	0–1

---

5. XGBoost và LightGBM

5.1. Giới thiệu

- **XGBoost (eXtreme Gradient Boosting, 2014):** Gradient Boosting được tối ưu cực mạnh về tốc độ, có sẵn regularization, xử lý ô trống tự động. Thống trị các cuộc thi Kaggle nhiều năm.
- **LightGBM (Microsoft, 2017):** Còn nhanh hơn nữa nhờ chia giá trị thành các "thùng" (histogram, giống bài 1.5) và mọc cây theo **lá** thay vì theo **tầng**. Rất hợp với dữ liệu lớn.
- **HistGradientBoosting:** Phiên bản lấy cảm hứng từ LightGBM, có sẵn trong Scikit-learn, không cần cài thêm.

Cả ba đều **có sẵn trên Google Colab**. Trên máy riêng: `pip install xgboost lightgbm`.

5.2. So sánh trên dữ liệu giá nhà

```
import time
import xgboost as xgb
import lightgbm as lgb
from sklearn.ensemble import RandomForestRegressor, HistGradientBoostingRegressor

def chay(ten, mh):
    t0 = time.time(); mh.fit(Xtr, ytr); giay = time.time() - t0      # đo thời
gian huấn luyện
    p = mh.predict(Xte)
```

```
    print(f"{ten:<24} MAE = {mean_absolute_error(yte, p):5.1f} | R² =
{r2_score(yte, p):.3f} | {giay:5.1f} giây")

chay("RandomForest (300)", RandomForestRegressor(n_estimators=300,
random_state=42, n_jobs=-1))
chay("GradientBoosting (500)", GradientBoostingRegressor(n_estimators=500,
learning_rate=0.1, max_depth=4, random_state=42))
chay("HistGradientBoosting", HistGradientBoostingRegressor(max_iter=500,
learning_rate=0.1, random_state=42))
chay("XGBoost", xgb.XGBRegressor(n_estimators=500, learning_rate=0.1, max_depth=6,
                                 subsample=0.8, colsample_bytree=0.8,
random_state=42))
chay("LightGBM", lgb.LGBMRegressor(n_estimators=500, learning_rate=0.1,
num_leaves=31,
                                   random_state=42, verbose=-1))       #
verbose=-1: tắt log
```

Kết quả mẫu (thời gian phụ thuộc máy):

```
RandomForest (300)       MAE =  29.1 | R² = 0.793 |   2.9 giây
GradientBoosting (500)   MAE =  28.4 | R² = 0.809 |  12.7 giây
HistGradientBoosting     MAE =  27.6 | R² = 0.822 |   0.3 giây
XGBoost                  MAE =  26.9 | R² = 0.827 |   0.9 giây
LightGBM                 MAE =  26.8 | R² = 0.831 |   0.3 giây
```

- **LightGBM** tốt nhất (R^2 0,831) và nhanh nhất, nhanh hơn `GradientBoosting` gốc **hơn 40 lần**.
- Từ đầu Giai đoạn 2: MAE giảm từ 77,6 (mốc) → 48,7 (Linear) → 29,1 (Random Forest) → **26,8**.
- Với dữ liệu dạng bảng có từ vài nghìn dòng trở lên, Gradient Boosting thường là **lựa chọn mạnh nhất**, thường mạnh hơn cả mạng nơ-ron.

5.3. Early stopping: tự biết khi nào dừng

Đặt số cây thật lớn, tách một phần train làm **validation**, và dừng khi điểm trên validation **không cải thiện** sau N cây liên tiếp.

```
Xtr2, Xval, ytr2, yval = train_test_split(Xtr, ytr, test_size=0.2,
random_state=42)   # tách validation từ TRAIN
m = xgb.XGBRegressor(n_estimators=5000, learning_rate=0.05, max_depth=6,
                     subsample=0.8, colsample_bytree=0.8, random_state=42,
                     early_stopping_rounds=100,     # dừng nếu 100 cây liên tiếp
không cải thiện
                     eval_metric="mae")
m.fit(Xtr2, ytr2, eval_set=[(Xval, yval)], verbose=False)
print("Số cây tốt nhất:", m.best_iteration, "| MAE test:",
round(mean_absolute_error(yte, m.predict(Xte)), 1))
```

Kết quả mẫu:

```
Số cây tốt nhất: 1106 | MAE test: 26.8
```

Mô hình tự dừng ở khoảng 1.100 cây thay vì chạy đủ 5.000. Không cần đoán số cây. **Luôn dùng early stopping với boosting.**

> ⚠️ Không dùng tập **test** làm `eval_set`: đó là rò rỉ dữ liệu (bài 1.11). Dùng validation tách từ train.

---

6. Phân loại và cột chữ: LightGBM tự xử lý

LightGBM và HistGradientBoosting có thể nhận **trực tiếp** cột chữ (kiểu `category` của Pandas), không cần one-hot.

```
from sklearn.datasets import fetch_openml
from sklearn.metrics import roc_auc_score
from sklearn.ensemble import HistGradientBoostingClassifier

credit = fetch_openml("credit-g", version=1, as_frame=True).frame
credit["vo_no"] = (credit["class"] == "bad").astype(int)
Xc = credit.drop(columns=["class", "vo_no"])              # cột chữ của OpenML đã
có kiểu category
yc = credit["vo_no"]
Xctr, Xcte, yctr, ycte = train_test_split(Xc, yc, test_size=0.2, random_state=42,
stratify=yc)

mc = lgb.LGBMClassifier(n_estimators=300, learning_rate=0.03, num_leaves=15,
min_child_samples=20,
                        subsample=0.8, subsample_freq=1, colsample_bytree=0.8,
                        random_state=42, verbose=-1).fit(Xctr, yctr)
print("LightGBM AUC:", round(roc_auc_score(ycte, mc.predict_proba(Xcte)[:, 1]),
3))

hc = HistGradientBoostingClassifier(learning_rate=0.05, max_iter=300,
max_leaf_nodes=15,
                                    categorical_features="from_dtype",
random_state=42).fit(Xctr, yctr)
print("HistGradientBoosting AUC:", round(roc_auc_score(ycte,
hc.predict_proba(Xcte)[:, 1]), 3))
```

Kết quả mẫu:

```
LightGBM AUC: 0.786
HistGradientBoosting AUC: 0.79
```

Bất ngờ: Trên dữ liệu tín dụng, boosting (0,79) **không thắng** Logistic Regression (0,803) hay Random Forest (0,799). Với chỉ **800 mẫu train**, mô hình mạnh khó phát huy, và chênh lệch vài phần nghìn trên 200 mẫu test

chủ yếu là may rủi. Bài học: **mô hình tốt nhất phụ thuộc vào dữ liệu**. Luôn thử nhiều mô hình và đánh giá cẩn thận (bài 2.17).

---

7. Khi nào dùng mô hình nào?

Tình huống	Nên thử đầu tiên
Cần giải thích rõ từng hệ số, dữ liệu ít	Linear / Logistic Regression
Dữ liệu bảng, muốn mạnh mà ít chỉnh	Random Forest
Dữ liệu bảng, muốn **mạnh nhất**	**LightGBM / XGBoost** (+ early stopping)
Văn bản, đặc trưng rất nhiều và thưa	Naive Bayes, Logistic Regression
Ảnh, âm thanh, văn bản dài	Deep Learning (Giai đoạn 3)

---

8. Bài tập

1. Tiếp tục **tính tay** ví dụ golf ở mục 2: nếu dùng learning rate = 1 (thay vì 0,5), dự đoán sau bước 1 là bao nhiêu? MSE bao nhiêu? Vì sao learning rate = 1 có thể nguy hiểm với dữ liệu thật?
2. Huấn luyện LightGBM cho giá nhà với early stopping (gợi ý: `lgb.LGBMRegressor(n_estimators=5000, ...)`, `fit(..., eval_set=[(Xval, yval)], callbacks=[lgb.early_stopping(100)])`). Số cây tốt nhất là bao nhiêu?
3. Thử XGBoost ở mục 5.2 với `max_depth` = 3, 6, 10. MAE test thay đổi thế nào?
4. Dùng `mc.feature_importances_` của LightGBM tín dụng (mục 6). Cột nào quan trọng nhất? So sánh với permutation importance ở bài 2.10.

Đáp án

> Mẹo Notion: chọn tiêu đề "Đáp án" → `Turn into → Toggle heading 3`.

1. Với η = 1: x = 1..4: 24,67 − 8,67 = **16,0**; x = 5, 6: 24,67 + 17,33 = **42,0**. Phần dư: −6; −4; 4; 6; −2; 2 → MSE = (36 + 16 + 16 + 36 + 4 + 4) / 6 ≈ **18,67** (nhỏ hơn 56,22 với η = 0,5). Nhanh hơn trên train, nhưng với dữ liệu thật có nhiễu, sửa "toàn bộ" sai số mỗi bước nghĩa là **học cả nhiễu** → overfit, như bảng ở mục 3.2 (learning_rate = 1,0 cho test kém nhất).
2. Code:

```
m = lgb.LGBMRegressor(n_estimators=5000, learning_rate=0.05, num_leaves=31,
random_state=42, verbose=-1)
m.fit(Xtr2, ytr2, eval_set=[(Xval, yval)], eval_metric="l1", callbacks=
[lgb.early_stopping(100, verbose=False)])
print(m.best_iteration_, round(mean_absolute_error(yte, m.predict(Xte)), 1))
```

Ghi lại số cây và MAE. Kết quả thường tương đương XGBoost. 3. `max_depth` nhỏ → mỗi cây yếu, cần nhiều cây hơn. `max_depth` lớn → mỗi cây mạnh, dễ overfit hơn. Với cùng 500 cây và learning_rate 0,1, độ sâu 6–8 thường là điểm cân bằng; ghi lại kết quả thật. 4. Code:

```
print(pd.Series(mc.feature_importances_,
index=Xc.columns).sort_values(ascending=False).head(6))
```

Mặc định LightGBM đếm **số lần** mỗi cột được dùng để chia, nên thiên vị cột số có nhiều giá trị (`credit_amount`, `age`, `duration`), giống Gini importance ở bài 2.10. Dùng `importance_type="gain"` khi tạo mô hình, hoặc permutation importance, để có bức tranh đáng tin hơn.

---

9. Flashcard

Mặt trước	Mặt sau
Ý tưởng Boosting?	Các cây nối tiếp, cây sau học sai số (phần dư) của các cây trước.
Boosting vs Random Forest?	RF: cây độc lập, sâu, lấy trung bình, giảm variance. Boosting: cây nối tiếp, nông, cộng dồn, giảm bias.
Mỗi cây trong Gradient Boosting học gì?	Phần dư (hướng ngược gradient của loss).
Learning rate trong boosting?	Mỗi cây chỉ sửa một phần sai số; nhỏ → cần nhiều cây hơn, ít overfit hơn.
Thêm cây vô hạn trong boosting thì sao?	Sẽ overfit (khác Random Forest).
Early stopping là gì?	Dừng thêm cây khi điểm validation không cải thiện sau N cây.
Dùng tập nào cho early stopping?	Validation tách từ train, không phải test.
XGBoost vs LightGBM?	Cùng là boosting tối ưu; LightGBM thường nhanh hơn nhờ histogram và mọc cây theo lá.
Mô hình mạnh nhất cho dữ liệu bảng thường là?	Gradient Boosting (LightGBM/XGBoost).
LightGBM có cần one-hot?	Không bắt buộc; nhận trực tiếp cột kiểu category.

Bài tiếp theo: 2.12 – SVM (Support Vector Machine).